

**MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE REPUBLIC OF
KAZAKHSTAN
NON-PROFIT JSC “L.N.GUMILYOV EURASIAN NATIONAL UNIVERSITY”
DEPARTMENT OF “COMPUTER AND SOFTWARE ENGINEERING”**

**METHODOLOGICAL GUIDELINES FOR PERFORMING LABORATORY EXERCISES
FOR THE DISCIPLINE INFORMATION AND CODING THEORY**

**for students enrolled in the educational program
6B06104 - Computer Engineering and Software**

Astana 2025

**Guidelines for Laboratory Work in the Course “Information and coding theory” for
Students Enrolled in the Educational Program 6B06104 - Computer Engineering and Software**

Developer: _____ **A. Turginbayeva**

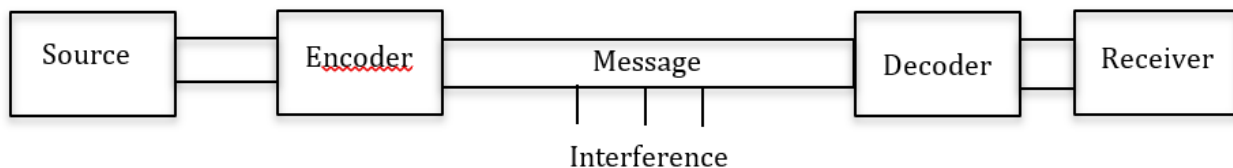
Recommended for publication and reviewed at a department meeting, protocol No. 11 dated
June 4, 2025

Department Chair _____ **K. Dyussekeyev**

LABORATORY EXERCISE No. 1

Abstract Alphabet

In a communication channel, a message composed of symbols (letters) from one alphabet can be converted into a message composed of symbols (letters) from another alphabet. The rule describing the one-to-one correspondence of letters between alphabets during such a conversion is called a code. The procedure of converting a message is called transcoding. Such a conversion of a message can take place when the message is transmitted from the source to the communication channel (encoding) and when the message is received by the recipient (decoding). We will refer to the devices that perform encoding and decoding as the encoder and decoder, respectively. Figure 1 shows a diagram illustrating the process of message transmission in the case of transcoding, as well as the effects of interference (see the next section).



Let's look at some examples of codes.

1. The Russian version of Morse code (the Morse alphabet is assigned to an alphabet composed of Russian uppercase letters and Arabic numerals):

A · –	J · – – –	S · · ·	2 · · – – –
B – · · ·	K – · –	T –	3 · · · – –
C – · – ·	L · – · ·	U · · –	4 · · · · –
D – · ·	M – –	V · · · –	5 · · · · ·
E ·	N – ·	W · – –	6 · · · · –
F · · · ·	O – – –	X – · · –	7 – – · · ·
G – – ·	P · – – ·	Y – · – –	8 – – – · ·
H · · · ·	Q – – – –	Z – · · ·	9 – – – – ·
I · ·	R · · ·	1 · – – – –	0 – – – – –

2. Trisime code (Latin alphabet characters are assigned combinations of three symbols: 1, 2, 3):

A 111	D 121	G 131	J211	M221	P231	S311	V321	Y331
B 112	E 122	H 132	K212	N222	Q232	T312	W322	Z332
C 113	F 123	I 133	L213	O223	R233	U313	X323	.333

The Trisime code is an example of a so-called uniform code (one in which all code combinations contain the same number of characters —in this case, three). An example of a non-uniform code is Morse code.

3. The ASCII (extended) codes are shown in Fig. 2.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0																
1	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
2	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
3	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
4	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
5	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
6	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
7	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
8	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
9	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
A	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
B	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
C	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
D	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
E	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
F	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

Fig. 2. ASCII codes (extended).

Independent work:

1. Write a program in Delphi 7 that demonstrates the process of encoding and decoding text in Latin (ASCII code, Trisime code) and in Russian (Morse code). A

Review Questions:

- 1) How is an alphabet defined?
- 2) What is a uniform or non-uniform code?
- 3) How can the high noise immunity of transmitted messages written in Russian be explained?

LABORATORY WORK №2

Binary-Decimal Code (BDC)

In the Binary-Decimal Code (BDC), each decimal digit is represented by a group of digits consisting of 4 binary symbols. This group allows us to form $N = 2^4 = 16$ code combinations. Since only ten digits are used in the decimal system, six combinations are redundant. Therefore, the choice of ten combinations used to construct the BDC has $16! \cdot 6! = 2.9 \cdot 10^{10}$ options. Using, for example, the first four powers of the number 2 ($2^0 = 1$; $2^1 = 2$; $2^2 = 4$; $2^3 = 8$) leads to one of the possible codes — 8-4-2-1.

Each BDC digit has a fixed weight. BDC codes are built under the following conditions:

1. The weight of the least significant digit q_1 is 1.

2. The weight of the second digit q_2 is 1 or 2.
3. The weights of the two remaining digits are chosen so that their sum is ≥ 6 (if $q_2 = 2$), or ≥ 7 (if $q_2 = 1$).

Based on this, 17 types of codes can be obtained:

8-4-2-1,	7-4-2-1,	6-4-2-1,	5-4-2-1,	4-4-2-1,
7-3-2-1,	6-3-2-1,	5-3-2-1,	4-3-2-1,	3-3-2-1,
6-2-2-1,	5-2-2-1,	4-2-2-1,	6-3-1-1,	5-3-1-1,
4-3-1-1,	5-2-1-1,			

To convert a single decimal digit into binary-decimal code, starting with the most significant binary digit, check whether the weight of the current (binary) digit is greater than the remainder of the number; if it is less than or equal to it, then write 1 in the output code and subtract the weight of the binary digit from the remainder of the number, otherwise write 0 in the output code.

Example of encoding

Encode digit 5 with 6-4-2-1 code.

1. Weight of the first bit = $6 > 5 \rightarrow$ first bit = 0.
2. Weight of the second bit = $4 \leq 5 \rightarrow$ bit = 1, remainder = 1.
3. Weight of the third bit = $2 > 1 \rightarrow$ bit = 0.
4. Weight of the fourth bit = $1 = \text{remainder} \rightarrow$ bit = 1.

Result: 0101.

To obtain the value of a tetrad (4 bits) written in binary-decimal code, it is necessary to sum the values of the weights of the digits whose values are equal to 1.

Example of decoding

Decode 0101 in 6-4-2-1 BDC:

$$0 \cdot 6 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 5.$$

BDCs are not unique in representing decimal numbers (except for code 8-4-2-1). Thus, code 4-3-2-1 allows the number 64 to be written as 1010 1000 or 0111 0101.

Example for encoding. 1. Encode the number 8701 using the binary-decimal code 5-2-1-1:

$$8 = 5 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 + 1 \cdot 0;$$

$$7 = 5 \cdot 1 + 2 \cdot 1 + 1 \cdot 0 + 1 \cdot 0;$$

$$0 = 5 \cdot 0 + 2 \cdot 0 + 1 \cdot 0 + 1 \cdot 0;$$

$$1 = 5 \cdot 0 + 2 \cdot 0 + 1 \cdot 1 + 1 \cdot 0,$$

The encoded number will look like this: 1110 1100 0000 0010.

Example for decoding. 1. Decode the number 001001011100, if it is known that it is encoded in the 6-3-2-1 binary-decimal code:

$$6 \cdot 0 + 3 \cdot 0 + 2 \cdot 1 + 1 \cdot 0 = 2;$$

$$6 \cdot 0 + 3 \cdot 1 + 2 \cdot 0 + 1 \cdot 1 = 4;$$

$$6 \cdot 1 + 3 \cdot 1 + 2 \cdot 0 + 1 \cdot 0 = 9, \text{ then the decoded number will be } 249.$$

When recording results in binary code, spaces are insignificant characters.

Independent work (tasks)

variant	Tasks
1	Encode number 8701 with 5-2-1-1 BDC
	Decode 001001011100 if encoded with 6-3-2-1 BDC
	Decode 100101110010 if encoded with 8-4-2-1 BDC
2	Decode 010111110010 if encoded with 4-3-2-1 BDC
	Encode number 927 with 7-4-2-1 BDC
	Encode number 331 with 7-3-2-1 BDC
3	Encode number 456 with 4-4-2-1 BDC
	Decode 000110101101 if encoded with 5-3-2-1 BDC
	Decode 010111010001 if encoded with 5-4-2-1 BDC
4	Decode 111100100010 if encoded with 3-3-2-1 BDC
	Encode number 904 with 6-2-2-1 BDC
	Encode number 805 with 6-4-2-1 BDC
5	Decode 100101111001 if encoded with 6-3-1-1 BDC
	Decode 101100110010 if encoded with 5-3-1-1 BDC
	Encode number 110 with 4-3-1-1 BDC
	Encode number 876 with 5-2-2-1 BDC

Review questions:

- 1) What is a binary-decimal code?
- 2) Under what conditions are BDCs constructed?
- 3) Where can BDCs be applied?

LABORATORY WORK №3

Gray Code

Reflected (reflex) codes are constructed in such a way that neighboring code combinations, unlike simple binary codes, differ by only one digit, i.e., the code distance between neighboring code combinations of such a code is equal to one. Of the reflected codes, the Gray code has become the most widely used.

The conversion of a simple binary code to a Gray code is performed according to the algorithm:

$Y_n = X_n$; $Y_i = X_i \text{ xor } X_{i+1}$, (xor – modulo 2 addition), where Y_i – value of the i-th digit of the Gray code; X_i , X_{i+1} – corresponding values of binary number digits ($i = 1, 2, \dots, n$, counting, digit with $i = 1$ is the least significant, $i = n$ is the most significant). That is, for all bits except the least significant bit, the Gray code value is equal to the sum modulo 2 of the corresponding and next higher bits of the binary code. The most significant bit is preserved.

Decoding (reverse conversion) of Gray code into binary code is performed using the following formula: $X_n = Y_n$; $X_i = X_{i+1} \text{ xor } Y_{i+1}$, where X_n and Y_n – values of the most significant bit of the binary code and Gray code, respectively ($i = 1, 2, \dots, n$, counting, digit with $i = 1$ is the least significant, $i = n$ is the most significant).

The most significant digit is transferred to the decoded sequence without changes.

The following bits are obtained by summing modulo 2 the corresponding and all more significant bits of the encoded message – for an 8-bit message, to decode the 4th bit, it is necessary to sum modulo 2 the bits 4, 5, 6, 7, and 8 of the code combination.

Example 1 (Encoding)

Encode binary number 10111101011 in Gray code.

$$Y_{11} = X_{11} = 1$$

$$Y_{10} = X_{10} \text{ xor } X_{11} = 1$$

$$Y_9 = X_9 \text{ xor } X_{10} = 1$$

$$Y_8 = X_8 \text{ xor } X_9 = 0$$

$$Y_7 = X_7 \text{ xor } X_8 = 0$$

$$Y_6 = X_6 \text{ xor } X_7 = 0$$

$$Y_5 = X_5 \text{ xor } X_6 = 1$$

$$Y_4 = X_4 \text{ xor } X_5 = 1$$

$$Y_3 = X_3 \text{ xor } X_4 = 1$$

$$Y_2 = X_2 \text{ xor } X_3 = 1$$

$$Y_1 = X_1 \text{ xor } X_2 = 0$$

Result: 11100011110.

Example 2 (Decoding)

Decode Gray-coded number 1110101.

$$X_7 = Y_7 = 1$$

$$X_6 = Y_7 \text{ xor } Y_6 = 0$$

$$X_5 = Y_7 \text{ xor } Y_6 \text{ xor } Y_5 = 1$$

$$X_4 = Y_7 \text{ xor } Y_6 \text{ xor } Y_5 \text{ xor } Y_4 = 1$$

$$X_3 = Y_7 \text{ xor } Y_6 \text{ xor } Y_5 \text{ xor } Y_4 \text{ xor } Y_3 = 0$$

$$X_2 = Y_7 \text{ xor } Y_6 \text{ xor } Y_5 \text{ xor } Y_4 \text{ xor } Y_3 \text{ xor } Y_2 = 0$$

$$X_1 = Y_7 \text{ xor } Y_6 \text{ xor } Y_5 \text{ xor } Y_4 \text{ xor } Y_3 \text{ xor } Y_2 \text{ xor } Y_1 = 0$$

Result: 1011001.

Independent work (tasks)

Encode a binary sequence into Gray code, and decode a Gray-coded sequence.

Variant	Assignments
1	1111101
	1011100
2	11000001
	10011101
3	1000011
	1111000
4	1010111
	100000101
5	110011001
	10001111001

Control questions

1. What are the features of the Gray code?
2. Where is the Gray code applied?
3. How is a binary code converted into Gray code?

LABORATORY WORK No. 4

Shannon-Fano Code

The basic principle underlying Shannon-Fano coding is that when selecting each digit of a code combination, one should strive to ensure that the amount of information it contains is as large as possible, i.e., regardless of the values of all previous digits, this digit should take both possible values (0 or 1) with equal probability, if possible. Of course, the number of digits in different designations varies, i.e., this code is uneven. Messages with a high probability correspond to short code combinations, while those with a lower probability correspond to longer code combinations.

Code combinations are constructed as follows:

1. Messages and their probabilities are recorded in a table and sorted in descending order of probability.

2. The table is divided into two parts so that the sums of probabilities in both parts are as close as possible. If there are two possible divisions with equally close sums of probabilities, the message that is different for them is assigned to the upper subgroup.

3. In the upper subtable, 0 is recorded as the most significant bit of the code word, and in the lower subtable, 1 is recorded.

4. The division of subtables according to paragraph 2 is repeated recursively until the final code combinations are obtained (the number of messages in the subtable will be equal to 1).

An example of constructing Shannon-Fano code combinations:

Given a set of messages with probabilities: $P_0 = 0.21$, $P_1 = 0.15$, $P_2 = 0.15$, $P_3 = 0.14$, $P_4 = 0.13$, $P_5 = 0.08$, $P_6 = 0.06$, $P_7 = 0.06$, $P_8 = 0.02$.

1. After the first division, we get two groups of messages: $P_0..P_2$ and $P_3..P_8$, since the sum of the probabilities of the messages in the first group is 0.51, and in the second, respectively, 0.49. The first digit of the code for the messages in the first group will be 0, and the second will be 1.

2. The first group is also divided into two: message P_0 and messages P_1 , P_2 . For the only message in the first group, we get the final code 00, and the digit 1 is added to the codes of messages P_1 and P_2 on the right. Similarly, group $P_3..P_8$ is divided into two: P_3 , P_4 and $P_5..P_8$.

3. For the pairs of messages P_1 and P_2 , P_3 and P_4 , final codes can be obtained. Group $P_5..P_8$ is divided into two more groups: P_5 and $P_6..P_8$. Although assigning message P_6 will not affect the redundancy of the code, in this implementation it must be assigned to the lower group.

4. The last group—messages $P_6..P_8$ —is first divided into P_6 and P_7 , P_8 , giving the final combination for P_6 , and then for P_7 and P_8 . Table 1

you can see the construction code combinations.

Table 1

$P_0 = 0.21$ $P_1 = 0.15$ $P_2 = 0.15$ $P_3 = 0.14$ $P_4 = 0.13$ $P_5 = 0.08$ $P_6 = 0.06$ $P_7 = 0.06$ $P_8 = 0.02$	$P_0 = 0.21$ $P_1 = 0.15$ $P_2 = 0.15$ $P_3 = 0.14$ $P_4 = 0.13$ $P_5 = 0.08$ $P_6 = 0.06$ $P_7 = 0.06$ $P_8 = 0.02$	$P_1 = 0.15$ $P_2 = 0.15$ $P_3 = 0.14$ $P_4 = 0.13$ $P_5 = 0.08$ $P_6 = 0.06$ $P_7 = 0.06$ $P_8 = 0.02$	$P_6 = 0.06$ $P_7 = 0.06$ $P_8 = 0.02$	$P_7 = 0.06$ $P_8 = 0.02$	P0 = 0.21 00 P1 = 0.15 010 P2 = 0.15 011 P3 = 0.14 100 P4 = 0.13 101 P5 = 0.08 110 P6 = 0.06 1110 P7 = 0.06 11110 P8=0.02 11111
--	--	--	--	------------------------------	---

Independent work:

Option No.	Tasks
1	1. Construct the optimal code for transmitting messages using a 32-letter equiprobable alphabet. What is the average length of a code word in the resulting codes?
2	1. Construct an optimal message code in which the probability of occurrence of letters is subject to the law $p_i = \left(\frac{1}{2}\right)^i$ t. e. letters of the message can be arranged in such a way that the probability of each of them appearing is half the probability of the next one appearing. 2. Determine the required bandwidth of the communication channel for transmitting messages using optimal codes constructed according to the Shannon-Fano method for an alphabet with the following probability distribution: $p_1 = 0.6$; $p_2 = 0.08$; $p_3 = 0.07$; $p_4 = 0.06$; $p_5 = 0.05$; $p_6 = 0.05$; $p_7 = 0.05$; $p_8 = 0.04$.
3	1. What is the length of the code combinations of the optimal code for transmitting messages composed of 16, 32, and 28 equally probable binary symbols? 2. Given an alphabet with the following probability distribution: $p_1 = 0.4$; $p_2 = 0.18$; $p_3 = 0.1$; $p_4 = 0.1$; $p_5 = 0.07$; $p_6 = 0.06$; $p_7 = 0.05$; $p_8 = 0.04$. Construct optimal codes using the Shannon-Fano method. Determine which code requires less communication channel capacity. 3. Determine the capacity of a binary communication channel for transmitting the optimal code constructed from the alphabet A, B, C, D, E, F, G, 3, if the probabilities of the letters appearing are equal to: 0.06; 0.15; 0.15; 0.07; 0.05; 0.3; 0.18; 0.04, and the transmission time is one letter – 0.1 sec.
5	1. Construct an optimal message code in which the probabilities of occurrence of letters of the primary alphabet consisting of 8 characters are integer negative

	power of two, and $\sum p_i = 1$
	2. Construct an optimal uneven code (OEC) using the Shannon-Fano method if the source message symbols appear with probabilities: $A_1 = A_2 = A_3 = A_4 = 0.19$; $A_5 = A_6 = A_7 = 0.08$. Compare how close the obtained codes are to the optimal ones.

Review questions:

- 1) What is the concept of the Shannon-Fano code?
- 2) How are code combinations constructed?
- 3) What is the basic principle of Shannon-Fano coding?

LABORATORY WORK No. 5

Huffman Code

Huffman code, which is similar to Shannon-Fano code, is even more convenient. The method of its construction is as follows. Messages are written in a column in descending order of probability. The last two messages are combined into one auxiliary message, which is assigned a total probability. The probabilities of the messages are again arranged in descending order of probability in an additional column, and the last two are combined. The process continues until we obtain a single message with a probability equal to one. Table 2 shows the method of constructing a code for six messages with probabilities of 0.4, 0.2, 0.2, 0.1, 0.05, and 0.05.

Example of table construction:

Table 2

Message No.		Auxiliary columns				
No.	Probability	1	2	3	4	5
1	0.4	0.4		0.4	0.6 } [→]	1
2	0.2	0.2	0.4	0.4 } [→]	0.4 } [→]	
3	0.2	0.2		0.2 } [→]		
4	0.1					
5	0.05 } [→]	0.1 } [→]	0.2 } [→]			
6	0.05 } [→]	0.1 } [→]	0.2 } [→]			

Independent work:

Option No.	Tasks
1	1. The letters <i>A</i> , <i>B</i> , and <i>C</i> appear in messages with probabilities of 0.7, 0.2, and 0.1, respectively. Determine the capacity of the communication channel required to transmit messages encoded by the ONK with two-character blocks if the source outputs , the symbols are generated at a rate of twenty characters per second.
2	1. Determine the required bandwidth of the communication channel for transmitting messages with optimal codes constructed using the Huffman method for an alphabet with the following probability distribution: $p_1 = 0.7$; $p_2 = 0.07$; $p_3 = 0.03$; $p_4 = 0.02$; $p_5 = 0.01$; $p_6 = 0.05$; $p_7 = 0.05$; $p_8 = 0.07$.
3	1. Given an alphabet with the following probability distribution: $p_1 = 0.2$; $p_2 = 0.15$; $p_3 = 0.5$; $p_4 = 0.03$; $p_5 = 0.07$; $p_6 = 0.06$; $p_7 = 0.14$; $p_8 = 0.18$; $p_9 = 0.01$; $p_{10} = 0.11$. Construct optimal codes using the Huffman method. Determine which code requires less communication channel capacity.
4	1. Write the first three code combinations of the optimal code for transmitting messages composed from the primary alphabet, containing 64 equally probable symbols.
	1. Determine the capacity of a binary communication channel for transmitting the optimal

5	code constructed from the alphabet L, G, N, P, D, B, T, X, if the probabilities of occurrence of the letters are equal to: 0.2; 0.3; 0.01; 0.09; 0.3; 0.03; 0.04; 0.03, respectively, and the transmission time for one letter is 0.1 <i>seconds</i> .
	2. Construct an ONC using the Huffman method, if the source symbols appear with probabilities: $A_1 = A_2 = A_3 = A_4 = 0.13$; $A_5 = A_6 = A_7 = 0.16$. Compare how close the obtained codes are to the optimal ones.

Review questions:

- 1) What is the concept of a Huffman code?
- 2) How are code combinations constructed in the Huffman code?
- 3) How is the probability table constructed?

LABORATORY WORK No. 6

Arithmetic encoding and decoding

The fundamental difference between this coding and the methods discussed earlier is its continuity and lack of need for blocking. The efficiency of arithmetic coding increases with the length of the compressed message (this is not the case for Huffman or Shannon-Fano coding). Although arithmetic coding usually provides better compression than Huffman coding, it is still rarely used in practice.

When compressing data, for example, from a file, all of the methods considered require two passes. The first is for collecting the frequencies of characters used as approximate values of character probabilities, and the second is for the compression itself.

An example of arithmetic coding. Let a discrete random variable (DRV) X take only two values, 0 and 1, with probabilities $2/3$ and $1/3$, respectively. Let us assign the value 0 to the interval $[0, 2/3]$ and 1 to the interval $[2/3, 1]$.

Then for the DRV $\vec{X}$

$$\dim(\vec{X}) = 3, H\vec{X} = H\vec{X}/3 = \log_2 3 - 2/3 \approx 0,9183 \text{ bits / sim (Table 3)}$$

Table 3 - Code construction.

Intervals and codes	Probability	Huffman code
$111 \left[\frac{26}{27}, 1 \right] \ni \frac{31}{32} = 0,11111$	1/27	0000
$11 \left[\frac{8}{9}, 1 \right] \ni \frac{31}{32} = 0,11111$	2/27	0001
$101 \left[\frac{22}{27}, \frac{8}{9} \right] \ni \frac{7}{8} = 0,111$	2/27	010
$1 \left[\frac{2}{3}, 1 \right] \ni \frac{3}{4} = 0,11$	4/27	001
$011 \left[\frac{16}{27}, \frac{2}{3} \right] \ni \frac{5}{8} = 0,101$	2/27	011
$01 \left[\frac{4}{9}, \frac{2}{3} \right] \ni \frac{1}{2} = 0,1$	4/27	100
$001 \left[\frac{8}{27}, \frac{4}{9} \right] \ni \frac{3}{8} = 0,011$	4/27	101
$0 \left[0, \frac{2}{3} \right] \ni \frac{1}{4} = 0,01$	8/27	11

$$ML_1(\vec{X}) = 65 / 81 \approx 0.8025 \text{ bits/symbol (arithmetic)}$$

$$ML_1(\vec{X}) = 76 / 81 \approx 0.9383 \text{ bits/sym. (block Huffman)}$$

$$ML_1(X) = ML(X) = 1 \text{ bit/sym. (Huffman)}$$

The original message is obtained from its arithmetic code using the following algorithm.

1. In the table for encoding DRV values, the interval containing the current code is determined – this interval uniquely determines one character of the original message. If this character is the end-of-message marker, then the end.

2. The lower boundary of the interval containing the current code is subtracted from the current code, and the difference is divided by the length of the same interval. The resulting number is considered the new current code value.

Independent work:

No. Option	Tasks
1	1. Create an arithmetic code with an end marker for the message BAABC
	2. Compose an arithmetic code for the message BABC received from DRV X with the following probability distribution $P(X=A)=1/4$, $P(X=B)=1/2$, $P(X=C)=1/4$. What will be the arithmetic code for the same message if X is distributed according to the law $P(X=A)=1/3$, $P(X=B)=7/15$, $P(X=C)=1/5$.
2	1. Let's unpack the code 00101110010100111, knowing that the set of message symbols consists of A, B, C, E, where the last one is the end-of-message marker.
	2. Compose an arithmetic code with an end marker for the message ALGORITHM.
3	1. Create an arithmetic code with a marker end for the message CODINGTHEORY.
	2. Let's unpack the code 00101110010100111, knowing that the set of symbols in the message consists of A, B, C, where the last one is the end-of-message marker
4	1. Compile arithmetic code with a marker end for message AAABCAAABC
	2. Compose an arithmetic code for the message BACBA received from DRV X with the following probability distribution $P(X=A)=1/4$, $P(X=B)=1/2$, $P(X=C)=1/4$. What will be the arithmetic code for the same message if X is distributed according to the law $P(X=A)=1/3$, $P(X=B)=7/15$, $P(X=C)=1/5$.
5	1. Compile an arithmetic code with a marker end for message MESSAGES
	2. Given a set of symbols A, B, C. The compressed message is ACCBCAABCA. Let's introduce the end-of-message marker E.

LABORATORY WORK No. 7

Lempel-Ziv Encoding

Lempel-Ziv encoding has four types of algorithms: LZ77, LZ78, LZSS, and LZW. The LZ77 Lempel-Ziv encoding algorithm is based on the principle of dynamic dictionaries. We will briefly present this concept and illustrate it with simple examples.

The algorithm is based on four main ideas:

1. Each successive encoded sequence of characters is added to the previously encoded

characters in such a way that, together with them, it forms a decomposition of all transmitted and received information into non-matching phrases.

2. This decomposition is stored in memory and used later as a dictionary.

3. Encoding is performed using pointers to phrases from an already formed dictionary of phrases.

Example 1. Encode the string "КРАСНАЯ КРАСКА" using the LZ77 algorithm.

Dictionary (8)	Buffer (5)	Code
""	"КРАСН"	<0,0,'К'>
"К"	"РАСНА"	<0,0,'Р'>
"КР"	"АСНАЯ"	<0,0,'А'>
"КРА"	"СНАЯ"	<0,0,'С'>
"КРАС"	"НАЯ К"	<0,0,'Н'>
"КРАСН"	"АЯ КР"	<5,1,'Я'>
".КРАСНАЯ"	"КРАС"	<0,0,' '>
"КРАСНАЯ"	"КРАСК"	<0,4,'К'>
"АЯ КРАСК"	"А . . ."	<0,0,'А'>

In the last line, the letter "А" is not taken from the dictionary because it is the last one.

Example 2. Encode the string "КРАСНАЯ КРАСКА" using the LZSS algorithm.

Dictionary (8)	Buffer (5)	Code	Code length
""	"КРАСН"	0,'К'	9
"К"	"РАСНА"	0,'Р'	9
"КР"	"АСНАЯ"	0,'А'	9
"КРА"	"СНАЯ"	0,'С'	9
"КРАС"	"НАЯ К"	0,'Н'	9
"КРАСН"	"АЯ КР"	1<5,1>	7
".КРАСНАЯ"	"Я КРА"	0,'Я '	9
"КРАСНАЯ"	" КРАС"	0,' '	9
"КРАСНАЯ "	"КРАСК"	1<0,4>	7
"НАЯ КРАС"	"КА . . ."	1<4,1>	7
"АЯ КРАСК"	"А"	1<0,1>	7

Here, the length of the resulting code is $7 \cdot 9 + 4 \cdot 7 = 91$ bits.

Example 3. Encode the string "КРАСНАЯ КРАСКА" using the LZ78 algorithm and a dictionary of 16 phrases.

Input phrase (in the dictionary)	Code	Dictionary position
----------------------------------	------	---------------------

"K"	<0,'K'>	1
"P"	<0,'P'>	2
"A"	<0,'A'>	3
"C"	<0,'C'>	4
"H"	<0,'H'>	5
"АЯ"	<3,'Я'>	6
" "	<0,' '>	7
"KP"	<1,'P'>	8
"AC"	<3,'C'>	9
"KA"	<1,'A'>	10

The length of the resulting code is $10 \cdot (4+8) = 120$ bits.

Example 4. Encoding the string "KPACHAJA KPACKA" using the LZW algorithm.
The dictionary size is 500 phrases.

Input phrase, wK (in the dictionary)	Code for w	Dictionary position
ASCII+		0-255
"KP"	0, 'K'	256
"PA"	0, 'P'	257
"AC"	0, 'A'	258
"CH"	0, 'C'	259
"HA"	0, 'H'	260
"АЯ"	0, 'А'	261
"Я"	0, 'Я'	262
" K"	0, ' '	263
"KPA"	<256>	264
"ACK"	<258>	265
"KA"	0, 'K'	266
"A"	0, 'A'	

In this example, the length of the resulting code is $12 \cdot 9 = 108$ bits.

Independent work:

Option No.	Tasks
1	1. Encode the string "KPACHAJA KPACKA" using the LZ78 algorithm, using a dictionary of 16 phrases.
	2. Encode the message "AABCDAAACCCDBB", "CYBERNETICS", "BLUES BLUER BLUE", calculate the lengths in bits of the resulting codes, using the LZ77 algorithm (dictionary-12 bytes, buffer-4 bytes).
2	1. Encode the string "KPACHAJA KPACKA" using the LZW algorithm,
	2. Encode the message "AABCDAAACCCDBB", "CYBERNETICS," "BLUES BLUER BLUE", calculate the lengths in bits of the resulting codes, using the LZ78 algorithm (dictionary of 16 phrases).
	3. Unpack each message and calculate the length of each compressed message in bits. Message compressed with LZSS (dictionary -12 bytes, buffer - 4 bytes), - 0'A0'F'0'X1 (9,2)1(8,2)1(6,3)1(4,4)1(9,1).
	1. Encode the string "KPACHAJA KPACKA", dictionary size -500 phrases.

3	2. Encode the message "AABCDAAACCCDBB", "CYBERNETICS", "BLUES BLUER BLUE", calculate the lengths in bits of the resulting codes using algorithms, LZSS (dictionary-12 bytes, buffer-4 bytes).
4	1. Encode the string "КРАСНАЯ КРАСКА" using the LZSS algorithm. 8-dictionary, 5-buffer. 2. Unpack each message and calculate the length of each compressed message in bits. Message compressed with LZ78 (dictionary -16 phrases), - (0, 'A') (0, 'F') (0, 'X') (1, 'F') (2, 'X') (5, 'A') (3, 'A') (2, 'F') (0, 'A') .
5	1. Unpack each message and calculate the code length of each compressed message in bits. Message compressed by LZ77 (dictionary -12 bytes, buffer - 4 bytes), - (0,0, 'A') (0,0, 'F') (0,0, 'X') (9,2, 'F') (8,1, 'F') (6,2, 'X') (4,3, 'A') . 2. Encode the message "AABCDAAACCCDBB", "CYBERNETICS", "BLUES BLUER BLUE", calculate the lengths in bits of the resulting codes, using the LZW algorithms (ASCII+ dictionary and 16 phrases).

Review questions:

- 1) Lempel-Ziv coding algorithms?
- 2) What is the main idea behind the LZ77 algorithm?
- 3) Can the LZ78 code be shorter than the LZW code for the first character with identical dictionary sizes?

LABORATORY WORK No. 8

Correlation code. Inverse code. Detection and correction of errors in messages.

A **correlation code** is a type of repeating code characterized by the introduction of additional symbols for each bit of the information part. If the bit in the information part is 0, then in the correlation code this bit is written as 01, if it is 1, then as 10. For example, the combination 10101 will be represented as 1001100110. The code distortion indicator is the appearance of combinations of the form 00 or 11 in "paired" elements.

The characteristics of the correlation code completely coincide with the characteristics of the simple repetition code. The difference in noise immunity of coded signals is detected only for asymmetric channels, for which probabilities transitions $0 \rightarrow 1$ and $1 \rightarrow 0$ are different, as well as for channels and symbols compared to other possible distortions.

Inverse code. A special type of code with repetition is the inverse code. The difference between it and the code with simple repetition is that in cases where the initial combination contains an even number of ones, the second combination exactly reproduces the initial one. If the original combination contains an odd number of ones, the repetition occurs in inverted form. For example, the combinations 01100 and 10110 are represented by the inverse code as 0110001100 and 1011001001.

The code combination is checked in the following sequence. First, the units of the first combination are summed. If their number is even, the elements of the additional combination are accepted unchanged. After that, both combinations are compared element by element, and if at least one mismatch is found, the combination is rejected. If the number of ones in the first combination is even, they are accepted in inverted form. Then, as before, the first and second combinations are compared element by element.

Example. Construct an inverse code for transmitting seven messages and determine its characteristics. Number of information characters $k = \log_2 N_p \approx 3$. We construct the generating matrix:

$$G = \begin{bmatrix} 100011 \\ 010101 \\ 001110 \end{bmatrix}$$

By summing the rows of the generating matrix in all possible combinations, we determine the remaining code vectors of the inverse code:

$$V_1 = 100011; V_2 = 010101; V_3 = 001110; V_4 = 110110; V_5 = 101101; V_6 = 011011; V_7 = 111000;$$

The distribution of code distances for the code vector is as follows

$$V_1 \ V_2 \ V_3 \ V_4 \ V_5 \ V_6 \ V_7$$

$$0 \ 4 \ 4 \ 3 \ 3 \ 3 \ 4$$

Hence $N_p(1) = N_p(2) = N_p(5) = N_p(6) = 0$; $N_p(3) = 3$; $N_p(4) = 3$. False transition coefficients false transitions from

$$K_{\pi}(1) = K_{\pi}(2) = K_{\pi}(5) = K_{\pi}(6) = 0; K_{\pi}(3) = \frac{3}{C_6^3} = 0.15; K_{\pi}(4) = \frac{3}{C_6^4} = 0.2.$$

Thus, this code has $d_{min} = 3$ and detects all 1-, 2-, 5-, 6-errors, 85% of triple errors, and 80% of quadruple errors.

Detection and correction of errors in messages.

In order for an error to be detected in a received message, the message must contain some redundant information that allows the erroneous code to be distinguished from the correct one.

The received message can consist of a code and its inversion. The code and inversion are sent to the communication channel as a single unit. An error at the receiving end is detected by comparing the code and its inversion. In uniform block code, code combinations with a constant ratio of zeros and ones in each code combination are considered valid. Such codes are called constant-weight codes. For binary codes, the number of code combinations in constant-weight codes with a length of n characters is equal to

$$N = C_n^l = \frac{n!}{l!(n-l)!}$$

where l is the number of units in the code word.

In general, to detect r errors, the minimum code distance

$$d_0 = r + 1.$$

The minimum code distance required for simultaneous error detection and correction is $d_0 = r + s + 1$,
 s is the number of correctable errors.

Example. Construct a generator matrix for a group code with a code distance $d_0 = 3$, capable of transmitting 100 different messages and having the potential to correct as many errors as possible.

Solution: to transmit 100 messages, the following inequalities must be satisfied $100 < 2^n_u$, from $n_u = 7$. Where $n_u = 7$, and $d_0 = 3$

$$n_k = [\log_2\{(n_u + 1) + [\log_2(n_u + 1)]\}] = 4$$

Thus, the number of rows in the generating matrix $n_u = 7$, the number of columns is $n_u + n_k = 7 + 4 = 11$. Since the number of correction bits is 4 and then, as rows of the test matrix P , we sequentially write out four-digit binary combinations with a weight $W_{(P)} = 4, 3, 2, 1$ (1111, 1110, 1101, 1011, 0111, 1100, 1010, 1001, 0110, 0101, 0011) and use those combinations in which there are more ones.

Since $n_u = 7$, we select 7 combinations. The check matrix should be constructed in such a way that the number of ones in the columns of matrix P is possible, since the number of checks performed during error correction depends on the number of ones in the column of matrix P .

The matrix of the densely packed code (11;7) has the following form:

Independent work:

No. Option	Assignments
1	1. Determine the minimum code distance required to detect a triple error in the code.
	2. What are the initial code combinations if the system operates with a triple repeat of the original signal, and the combinations received are: 11110, 00110, 11001, 10101, 11001, 01101, 11101, 11110, 11000.
2	1. How many errors can be corrected by a code in which the minimum code distance between code combinations is observed $d_0 = 7$ is maintained between code combinations?
	2. Determine the correction capabilities of the following code: 001, 010, 111.
	3. Add zeros and ones to the words below so that the parity check could detect any single bit error: 111011, 111010, 111000, 111100, 110111, 111101, 011001, 110000.
3	1. Given the code words: 00001, 11100, 10110, 01110. Is it possible to detect a single error in them?
	2. Construct a four-digit binary code that detects a single error.
4	1. What is the code distance between the combinations 0001 and 0001, 11000111001, and 10000011101?
	2. The source alphabet consists of the letters A, B, C, which correspond to the following binary code combinations: A- 00000, B-00111, C-11100. Determine whether any single error in any of the transmitted binary codes.

$$C_{(11,7)} = \begin{pmatrix} 10000001111 \\ 01000001110 \\ 00100001101 \\ 00010001011 \\ 00001000111 \\ 00000010110 \end{pmatrix}$$

5	1. Is the code capable of correcting errors if its combinations are as follows: 1001010, 0101110, 1101001, 0011011, 1011100.
	2. Construct a generating matrix for a group linear code that corrects the maximum possible number of error variants with multiplicity $r+1$, $r+2$, etc. when transmitting fifteen-bit binary information combinations.
	3. Construct a generator matrix for a group code with $d_0=3$ that satisfies the condition of maximum simplicity of the encoder and decoder when transmitting 1000 messages.

Review questions:

- 1) Can you describe the characteristics of the correlation code?
- 2) How does an inverse code differ from a simple repetition code?
- 3) How is a code combination checked in an inverse code?

LABORATORY WORK No. 9

Hamming Code

Hamming code is one of the most common systematic codes. These include codes with a minimum code distance $d_{min} = 3$, which correct all single errors.

The formation of r check elements in the combination of this code is carried out according to k information elements. Thus, the length of the code combination is $n = r + k$. Check elements are linear combinations of information elements, i.e., weighted sums of information elements with weight coefficients 1 and 0. For binary codes, addition modulo 2 is used as a linear operation. The subtraction operation coincides with the addition operation.

The sequence of 0s and 1s in a code combination is also called a code vector. The Hamming code has the properties of linear codes:

1. Sum or difference of code combinations of the code gives a vector belonging to a given code.
2. Linear codes form an algebraic group with respect to the operation of addition modulo 2.
3. The minimal code distance between code vectors of a group code is equal to the minimum weight of non-zero code vectors.

When transmitting a code vector, any element can be distorted, and the number of such situations is equal to $C_n^1 = n$. To this should be added one more situation when no error occurred. Thus, the total number of check element combinations 2^r must exceed the number of possible error situations in the code, taking into account the absence of errors for their correct differentiation and determination of error locations: $2^r \geq n + 1$. Since $2^n = 2^{r+k} = 2^k 2^r$, we can write: $2^n \geq (n + 1) 2^k$, where 2^n is the total number of code combinations.

The minimum ratio of correction bits to information bits below which the code cannot retain its correction capabilities is determined from the expression $2^r - 1 = n$.

To calculate the basic parameters of the Hamming code, you can specify the number of check elements r , when n and $k = n - r$ are determined from the last expression. The ratios between r , n , and k for the Hamming code are given in Table 4:

Table 4.

r	2	3	3	3	3	4	4	4	4	4	4	4	4	5
n	3	4	5	6	7	8	9	10	11	12	13	14	15	16
k	1	1	2	3	4	4	5	6	7	8	9	10	11	11

A characteristic feature of the check matrix of a code with $d_{min} = 3$ is that its columns represent different non-zero combinations of length r . For example, when $r = 4$ and $n = 15$, the code (15, 11), the check matrix looks like this:

H _{15,11} =	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
	u ₁	u ₂	u ₃	u ₄	u ₅	u ₆	u ₇	u ₈	u ₉	u ₁₀	u ₁₁	u ₁₂	u ₁₃	u ₁₄	u ₁₅

So, if we take combinations of a simple four-element code and discard the zero combination, we can easily obtain a check matrix by writing all code combinations sequentially in the columns of matrix H .

By rearranging the columns containing one unit to the right, we get the following matrix:

H _{15,11} =	0	0	0	0	1	1	1	1	1	1	1	1	0	0	0
	0	1	1	1	0	0	0	1	1	1	1	0	1	0	0
	1	0	1	1	0	1	1	0	0	1	1	0	0	1	0
	1	1	0	1	1	0	1	0	1	0	1	0	0	0	1
	a ₁	a ₂	a ₃	a ₄	a ₅	a ₆	a ₇	a ₈	a ₉	a ₁₀	a ₁₁	a ₁₂	a ₁₃	a ₁₄	a ₁₅

In accordance with this matrix, we obtain a system of verification equations, which we use to calculate the verification digits:

$$\begin{aligned}
 b_1 &= a_5 (+) & a_6 (+) & a_7 (+) & a_8 (+) & a_9 (+) & a_{10} (+) & a_{11} \\
 b_2 &= a_2 (+) & a_3 (+) & a_4 (+) & a_8 (+) & a_9 (+) & a_{10} (+) & a_{11} \\
 b_3 &= a_1 (+) & a_3 (+) & a_4 (+) & a_6 (+) & a_7 (+) & a_{10} (+) & a_{11} \\
 b_4 &= a_1 (+) & a_2 (+) & a_4 (+) & a_5 (+) & a_7 (+) & a_9 (+) & a_{11}
 \end{aligned}$$

When an error occurs in the code combination, those check relations that include the value of the erroneous digit will not be satisfied.

Thus, if an error occurs in the 7th information digit, equations 1, 3, and 4 will not be satisfied, i.e., the syndrome is 1011 (coinciding with the 7th column of matrix H). Thus, the location of the column of matrix H that coincides with the calculated syndrome indicates the location of the error. The calculated syndrome value necessarily coincides with one of the columns of matrix H , since all possible binary r -bit numbers are selected as columns.

Hamming proposed arranging the columns of the check matrix so that the number of the i -th column of matrix H and the number of the erroneous digit of the code combination correspond to the binary representation of the number i . Then the syndrome obtained from the check equations will be the binary representation of the number of the digit of the code combination in which the error occurred

error occurred. For this, the check digits should not be at the end of the code combination, but at the position numbers that are expressed as powers of two (2^0 , 2^1 , 2^2 , ..., 2^{r-1}), as is the case in the untransformed matrix H, because each of them is included in only one of the check equations.

In the latter case, the parity bits are placed between the information bits. The syndrome, according to the untransformed parity matrix $H_{15,11}$, is determined from the equations:

$$\begin{aligned} S_1 &= u_1 \quad (+) u_3 \quad (+) u_5 \quad (+) u_7 \quad (+) u_9 \quad (+) u_{11} \quad (+) u_{13} \quad (+) u_{15} \\ S_2 &= u_2 \quad (+) u_3 \quad (+) u_6 \quad (+) u_7 \quad (+) u_{10} \quad (+) u_{11} \quad (+) u_{14} \quad (+) u_{15} \\ S_3 &= u_4 \quad (+) u_5 \quad (+) u_6 \quad (+) u_7 \quad (+) u_{12} \quad (+) u_{13} \quad (+) u_{14} \quad (+) u_{15} \\ S_4 &= u_8 \quad (+) u_9 \quad (+) u_{10} \quad (+) u_{11} \quad (+) u_{12} \quad (+) u_{13} \quad (+) u_{14} \quad (+) u_{15} \end{aligned}$$

The digits u_1 , u_2 , u_4 and u_8 , which occur once in this system of equations, are selected as check digits.

For example, if you need to encode a simple code message 11001010110 ($k=11$) in Hamming code, you need to determine the check digits in the combination: $u_1 \ u_2 \ 1 \ u_4 \ 100 \ u_8 \ 1010110$. From the check matrix, we obtain:

$$\begin{aligned} u_1 &= u_3 \quad (+) u_5 \quad (+) u_7 \quad (+) u_9 \quad (+) u_{11} \quad (+) u_{13} \quad (+) u_{15} \\ u_2 &= u_3 \quad (+) u_6 \quad (+) u_7 \quad (+) u_{10} \quad (+) u_{11} \quad (+) u_{14} \quad (+) u_{15} \\ u_4 &= u_5 \quad (+) u_6 \quad (+) u_7 \quad (+) u_{12} \quad (+) u_{13} \quad (+) u_{14} \quad (+) u_{15} \\ u_8 &= u_9 \quad (+) u_{10} \quad (+) u_{11} \quad (+) u_{12} \quad (+) u_{13} \quad (+) u_{14} \quad (+) u_{15} \end{aligned}$$

Calculating the check digits according to this system, we obtain the following for this message:

$$\begin{aligned} u_1 &= 1 (+) 1 (+) 0 (+) 1 (+) 1 (+) 1 (+) 0 = 1, \\ u_2 &= 1 (+) 0 (+) 0 (+) 0 (+) 1 (+) 1 (+) 0 = 1, \\ u_4 &= 1 (+) 0 (+) 0 (+) 0 (+) 1 (+) 1 (+) 0 = 1, \\ u_8 &= 1 (+) 0 (+) 1 (+) 0 (+) 1 (+) 1 (+) 0 = 1. \end{aligned}$$

Therefore, the Hamming code combination for the message in question will be 111110001010110.

Suppose that the 8th element of the code combination is received incorrectly, i.e., the message 111110011010110 is received. Calculating the syndrome, we get: $S_1 = 0$; $S_2 = 0$; $S_3 = 0$; $S_4 = 1$, i.e. the syndrome is $1000_2 = 8_{10}$ (obviously, to obtain the correct result, the syndrome bits must be written starting with the most significant one, i.e. $S_4 \ S_3 \ S_2 \ S_1$). Thus, the 8th digit of the received code combination must be corrected.

Independent work:

Option No.	Tasks
1	1. Construct a Hamming code for the information combination 0101. Show the error detection process.

	2. Determine the minimum code distance between the binary vectors: 1100011; 1001111; 1010101.
2	1. Construct a matrix for a group code capable of correcting a single error when transmitting 16 characters of the primary alphabet.
	2. Construct Hamming code layouts for detecting and correcting single errors: a) a three-digit binary code; b) a five-digit binary code.
3	1. What do the combinations of the Hamming correction code for transmitting messages 1101 and 1011? Show the process of constructing the correction code.
	2. Is the code capable of correcting errors if its combinations are as follows: 1001010;0101110;1101001;0011011;1011100?
4	1. What is the sum modulo 2 of the following code vectors: 1000111; 0100011; 1100100; 0010110?
	2. The following the following combinations in Hamming code Hamming code: 1101001,0001111,0111100. Received: 1001001, 0011111, 0110100. Show error detection process.
5	1. Construct a Hamming code for correcting a single error and detecting a double error in a four-digit binary code. Show the construction process.
	2. Determine the weight of the code vector obtained by adding the code vectors 1010001 and 0110101 modulo 2.

Review questions:

- 1) Which codes belong to the Hamming code?
- 2) What is the length of the Hamming code?
- 3) How is the syndrome determined?

LABORATORY WORK No. 10

Varshama Code. Error Detection

The code proposed by Varshamov is a typical representative of systematic codes, i.e., the sum of any permitted combinations is also a permitted combination. This makes it possible to construct all combinations of the code with only a limited number of them available. A systematic code is constructed on the basis of a generating matrix. The generating matrix can be represented as two submatrices: information $|E_{(k)}|$ (unit matrix, k – number of information elements) and verification $|C_{(rk)}|$.

Construction of matrices G and H .

The check matrix $|C_{rk}|$ for the Varshavsky code is constructed by selecting various combinations and must satisfy the following conditions:

1. Each row of the submatrix $|C_{rk}|$ must contain at least $d - 1$ units (d – minimum code distance);
2. The sum of any j rows must have at least $d - j$ units;
3. The number of columns in the submatrix (r – number of check elements)

$$2^r > 1 + \sum_{i=1}^{d-2} C_{n-1}^i \quad (1)$$

where n is the length of the code combination.

$$C_{n-1}^i = \frac{(n-1)!}{i!((n-1)-i)!} \quad (2)$$

The Varshamov code, like any other systematic code, is capable of detecting and correcting errors. The number of correctable errors:

$$t_u = \frac{d-1}{2} \quad (3)$$

t_u – integer, i.e. in (3) rounded down to the nearest integer.

In order to detect in which digit the error was made, a check matrix H is constructed. The check matrix consists of two submatrices: $|D_{kr}|$, containing k columns and r rows, and $|E_r|$ – a unit matrix. Each row $|D_{kr}|$ corresponds to a column of check digits of the submatrix $|C_{rk}|$ forming the matrix G .

General form of matrices G and H (example).

$$G = \begin{vmatrix} 1 & 0 & 0 & a_1 & a_2 \\ 0 & 1 & 0 & b_1 & b_2 \\ 0 & 0 & 1 & c_1 & c_2 \end{vmatrix}$$

$$H = \begin{vmatrix} a_1 & b_1 & c_1 & 1 & 0 \\ a_2 & b_2 & c_2 & 0 & 1 \end{vmatrix}$$

$$a_1 \dots c_2 = \overline{0,1}$$

Example 1. Encoding

1. Encoding using matrix G.

Let's encode 101. To do this, as can be seen from matrix G, we need to add the first and third rows modulo

2. We get 1 0 1 (a1 xor c1) (a2 xor c2).

3. Encoding using matrix H

$$H = \begin{bmatrix} a_1 & b_1 & c_1 & 1 & 0 \\ a_2 & b_2 & c_2 & 0 & 1 \end{bmatrix}$$

Let's encode 011. To do this, we will proceed as follows:

1. We calculate the first check element $b_1 = (0 \& a_1) \text{ xor } (1 \& b_1) \text{ xor } (1 \& c_1)$.

2. Calculate the second check element $b_2 = (0 \& a_2) \text{ xor } (1 \& b_2) \text{ xor } (1 \& c_2)$.

3. Let's write down the code 0 1 1 b_1 b_2 .

Error detection.

When using the check matrix H to detect errors, the code combination is checked on the receiving side using so-called CODE SYNDROMES (S), which are calculated as follows:

$$H = \begin{bmatrix} a_1 & b_1 & c_1 & 1 & 0 \\ a_2 & b_2 & c_2 & 0 & 1 \end{bmatrix}$$

Code: e_1 e_2 e_3 b_1 b_2

$S1 = (e_1 \& a_1) \text{ xor } (e_2 \& b_1) \text{ xor } (e_3 \& c_1) \text{ xor } b_1$
 $S2 = (e_1 \& a_2) \text{ xor } (e_2 \& b_2) \text{ xor } (e_3 \& c_2) \text{ xor } b_2$ where $e_1 \dots c_2$ are elements of matrix H.

There is no one-to-one correspondence between the syndrome combination and the error combination that caused it. Therefore, each syndrome is assigned a correctable error that is most likely to occur in the communication channel.

Example 2.

Let the length of the code combination be $n = 7$ and the minimum code distance be $d = 3$.

1. The number of columns G is 7.

2. The number of correctable by code errors

$$t_u = \frac{3-1}{2} = 1$$

3. Number of lines G equal to $k=n-r$; from (1) we find r :

$$2^r > 1 + \sum_{i=1}^{3-2} C_{6-1}^i \Rightarrow 2^r > 1 + \frac{6!}{1!(6-1)!} = 7 \Rightarrow 2^r > 7 \Rightarrow r = 3$$

From this, $k = 7 - 3 = 4$.

4. Let's construct the generating matrix G and the verification matrix H. G:

H:

5. Encoding using G Number:

1100

Code: 1100 110 (1st row xor 2nd row)

Number: 1010

Code: 1010 101 (2nd row xor 4th row)

Number: 0110

Code: 0110 011 (3rd row xor 4th row)

6. Error detection.

1. Let's calculate the syndromes for the code with an error in the first digit: Code: 1100 110

Code with error: 0100 110

$S1 = (0 \& 0) \text{ xor } (1 \& 1) \text{ xor } (0 \& 1) \text{ xor } (0 \& 1) \text{ xor } 1=0$

$S2 = (0 \& 1) \text{ xor } (1 \& 0) \text{ xor } (0 \& 1) \text{ xor } (0 \& 1) \text{ xor } 1=1$

$S3 = (0 \& 1) \text{ xor } (1 \& 1) \text{ xor } (0 \& 0) \text{ xor } (0 \& 0) \text{ xor } 0=1.$

2. Similarly, calculating the syndromes for codes with errors in other bits, we obtain the following table:

Number of distorted digit	Syndrome
1	0 1 1
2	1 0 1
3	1 1 0
4	1 1 1
5	1 0 0
6	0 1 0
7	0

3. Using this table, you can to the code, calculate the syndromes, and determine the number of the distorted digit.

Independent work:

Option No.	Tasks
1	1. Code length $n=10$ elements. Minimum code distance $d=6$. Given n and d , calculate: the number of columns in the generator matrix, the number of correctable errors, the number of rows in the generator matrix, the number of check digits?

	2. Construct the generator matrix G for tu=1 and d=3: G =		??????	?????	
			??????	?????	
			??????	?????	
			??????	?????	
			??????	?????	
			??????	?????	
2	1. Encode the number: 010101. G =	100000	0011		
		010000	0101		
		001000	1001		
		000100	0110		
		000010	1010		
		000001	1100		
	2. Code length n=17 elements. Minimum code distance d=3. Given n and d, calculate: the number of columns in the generator matrix, the number of correctable errors, the number of rows in the generator matrix, the number of parity bits?				
3	1. given generating matrix G , construct a test N:	100000	0011		
		010000	0101	??????	?????
		001000	1001	??????	?????
		000100	0110	??????	?????
		000010	1010	??????	?????
		000001	1100		
	2. Using the code syndrome table and matrix H, correct the errors in syndrome				
		0011			
		0101			
		1001			
		0110			
	message.1010			001011	1000
				010101	0100
				100110	0010
				111000	0001
		1100			
		1000			
		0100			
		0010			
		0001			
4	1. Code length n=13 elements. Minimum code distance d=3. Given n and d, calculate: the number of columns in the generator matrix, the number of correctable errors, the number of rows in the generator matrix, the number of parity bits?				

	2. Encode the number: 101110. $G =$	<table><tr><td>100000</td><td>0011</td></tr><tr><td>010000</td><td>0101</td></tr><tr><td>001000</td><td>1001</td></tr><tr><td>000100</td><td>0110</td></tr><tr><td>000010</td><td>1010</td></tr><tr><td>000001</td><td>1100</td></tr></table>	100000	0011	010000	0101	001000	1001	000100	0110	000010	1010	000001	1100	
100000	0011														
010000	0101														
001000	1001														
000100	0110														
000010	1010														
000001	1100														
5	1. Using the code syndrome table and matrix H, correct the errors in <i>syndrome</i> 0011 0101 1001 0110 message.1010 1100 1000 0100 0010 0001 $H =$<table><tr><td>001011</td><td>1000</td></tr><tr><td>010101</td><td>0100</td></tr><tr><td>100110</td><td>0010</td></tr><tr><td>111000</td><td>0001</td></tr></table> Received message: 1010010111	001011	1000	010101	0100	100110	0010	111000	0001						
001011	1000														
010101	0100														
100110	0010														
111000	0001														
	2. Code length n=14 elements. Minimum code distance d=6. Given n and d, calculate: the number of columns in the generator matrix, the number of correctable errors, the number of rows in the generator matrix, the number of check digits?														

Control questions:

1. What symbols does the generating matrix of the Varshavsky code consist of?
2. How is the submatrix of check symbols constructed?
3. What is the formula for determining the columns of the submatrix?

LABORATORY WORK No. 11

Reed-Mauler Codes

Reed-Mauler codes form a class of binary systematic codes with different redundancy values $R = r / n$ and minimum code distance values d_{min} . These codes are characterized by the following parameter values: code length $n = 2^m$; number of information bits:

$$k = \sum_{i=0}^b C_m^i$$

minimum code distance $d = 2^{m-b}$, where $m \geq 3$ – any positive integer; $b < m$ – code order.

The construction of Reed-Maurer codes boils down to the following. First, a generator matrix G is constructed, the first row of which contains n units. This is followed by m rows, which can be conveniently viewed as an $(m \times n)$ matrix, whose columns are binary numbers (starting from zero). It is convenient to count the digits of binary numbers from top to bottom. These m rows form the first-order vectors b . Next come the rows of second-order vectors, which are obtained from all products of two rows of the first order, then the rows of the third order, which are all products of three rows of the first order, and so on.

Thus, if a matrix G is constructed for a code of order b , it contains one row with all ones, m rows of the first order, C_m^2 rows of the second order, C_m^3 rows of the third order, and so on. In general, the number of rows is

and coincides with the number of information bits k .

$$C_m^0 + C_m^1 + C_m^2 + \dots + C_m^b = \sum_{i=0}^b C_m^i$$

Decoding Reed-Mauler codes is very simple. In these codes, each of information symbols uk_i can be described by at least $d = 2^{m-b}$ independent check equations, the arguments of which are the symbols u_i of the code word. Analysis of the results of solving these equations according to the majority principle allows the value of the information symbols to be determined. Thus, when correcting errors, the stage of determining the location of errors is eliminated.

The simplicity of the structure of the generator matrix of these codes allows establishing a connection between the information symbols uk_i and the symbols of the code word u_i . Each of the information symbols can be described by $d = 2^{m-b}$ equations.

In general, there must be at least *two* independent equations for each uk_i . The sums of which code word symbols should be equal to the information symbol $uk(i)$ can be determined as follows. Let us call the component corresponding to the j -th zero in the vector $x(i)$ and the component corresponding to the j -th one in the vector $x(i)$ paired components.

Example 1. Constructing a matrix

Construct a matrix for a second-order Reed-Maller code ($b = 2$) and $m = 3$ ($b < m$). The first row contains $n = 2^3$ units (11111111). We construct a first-order matrix (3×8):

0	1	2	3	4	5	6	7
0	1	0	1	0	1	0	1
0	0	1	1	0	0	1	1
0	0	0	0	1	1	1	1

To continue constructing strings, keep in mind that if strings are multiplied, for example, 01010101 and 00110011, the result is obtained by multiplying the strings bit by bit:

$$\begin{array}{r}
 01010101 \\
 * \\
 00110011 \\
 \hline
 0010001
 \end{array}$$

Next, the construction of the matrix boils down to obtaining second-order strings. To do this, we multiply the first and second strings, the first and third, then the second and third. As a result, we get the following three strings:

0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	1
0	0	0	0	0	0	1	1

A fully constructed matrix for a second-order Reed-Maller code with $m = 3$, whose eleven information symbols $uk_1, uk_2, uk_3, uk_4, uk_5, \dots, uk_{11}$ has the form:

G =	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	x ₀	Uk ₁
	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	x ₁	Uk ₂
	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1	x ₂	Uk ₃
	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1	x ₃	Uk ₄
	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	x ₄	Uk ₅
	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	x ₁ x ₂	Uk ₆
	0	0	0	0	0	1	0	1	0	0	0	0	0	1	0	1	x ₁ x ₃	Uk ₇
	0	0	0	0	0	0	0	0	0	1	0	1	0	1	0	1	x ₁ x ₄	Uk ₈
	0	0	0	0	0	0	1	1	0	0	0	0	0	0	1	1	x ₂ x ₃	Uk ₉
	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1	1	x ₂ x ₄	Uk ₁₀

																	4	
	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	$x_3 \times \frac{x}{4}$	Uk_{11}
	u_1	u_2	u_3	u_4	u_5	u_6	u_7	u_8	u_9	u_{10}	u_{11}	u_{12}	u_{13}	u_{14}	u_{15}	u_{16}		

Example 2. Encoding

The code vector (encoded information symbols) has the following form: V

$$= uk_1 x_0 + uk_2 x_1 + uk_3 x_2 + uk_4 x_3 + uk_5 x_4 + uk_6 x_1 x_2 + uk_7 x_1 x_3 + uk_8 x_1 x_4 + uk_9 x_2 x_3 + uk_{10} x_2 x_4 + uk_{11} x_3 x_4 = f(u_1, u_2, u_3, \dots, u_{16}).$$

From the given matrix, it follows that:

$u_1 = uk_1 ;$	$u_5 = uk_1 + uk_4 ;$	$u_9 = uk_1 + uk_5 ;$	$u_{13} = uk_1 + uk_4 + uk_5 + uk_{11}$
$u_2 = uk_1 + uk_2 ;$	$u_6 = uk_1 + uk_2 + uk_4 + uk_7 ;$	$u_{10} = uk_1 + uk_2 + uk_5 + uk_8 ;$	$u_{14} = uk_1 + uk_2 + uk_4 + uk_5 + uk_7 + uk_8 + uk_{11}$
$u_3 = uk_1 + uk_3 ;$	$u_7 = uk_1 + uk_3 + uk_4 + uk_9 ;$	$u_{11} = uk_1 + uk_3 + uk_5 + uk_{10} ;$	$u_{15} = uk_1 + uk_3 + uk_4 + uk_5 + uk_9 + uk_{10} + uk_{11}$
$u_4 = uk_1 + uk_2 + uk_3 + uk_6 ;$	$u_8 = uk_1 + uk_2 + uk_3 + uk_4 + uk_6 + uk_7 + uk_9 ;$	$u_{12} = uk_1 + uk_2 + uk_3 + uk_5 + uk_6 + uk_8 + uk_{10} ;$	$u_{16} = uk_1 + uk_2 + uk_3 + uk_4 + uk_5 + uk_6 + uk_7 + uk_8 + uk_9 + uk_{10} + uk_{11}$

Suppose that the original message is 10001011011. Then the encoded message (code vector) looks like 1111101001101100.

Example 3. Decoding.

Let our code be characterized by the following parameters: $m = 4$, $b = 2$. From these, we calculate the remaining indicators: $n = 2^m = 2^4 = 8$; $d = 2^{m-b} = 2^2 = 4$.

The message encoded with the Reed-Maurer code looks like this: 1111101001101100. Let's determine the original message. Using any of the ratios, we determine uk_i for second-order vectors:

$$\begin{aligned}
uk_{11} &= u_1 + u_5 + u_9 + u_{13} = u_2 + u_6 + u_{10} + u_{14} = u_3 + u_7 + u_{11} + u_{15} = u_4 + u_8 + u_{12} + u_{16} = 1 + 1 + 0 + 1 = 1; \\
uk_{10} &= u_1 + u_3 + u_9 + u_{11} = u_2 + u_4 + u_{10} + u_{12} = u_5 + u_7 + u_{13} + u_{15} = u_6 + u_8 + u_{14} + u_{16} = 1 + 1 + 1 + 0 = 1; \\
uk_9 &= u_1 + u_3 + u_5 + u_7 = u_2 + u_4 + u_6 + u_8 = u_9 + u_{11} + u_{14} + u_{16} = u_{10} + u_{12} + u_{14} + u_{16} = 0 + 1 + 1 + 0 = 1; \\
uk_8 &= u_1 + u_2 + u_9 + u_{10} = u_3 + u_4 + u_{11} + u_{12} = u_5 + u_6 + u_{13} + u_{14} = u_7 + u_8 + u_{15} + u_{16} = 1 + 1 + 0 + 1 = 1; \\
uk_7 &= u_1 + u_2 + u_5 + u_6 = u_3 + u_4 + u_7 + u_8 = u_9 + u_{10} + u_{13} + u_{14} = u_{11} + u_{12} + u_{15} + u_{16} = 1 + 1 + 1 + 0 = 1; \\
uk_6 &= u_1 + u_2 + u_3 + u_4 = u_5 + u_6 + u_7 + u_8 = u_9 + u_{10} + u_{11} + u_{12} = u_{13} + u_{14} + u_{15} + u_{16} = 1 + 1 + 1 + 1 = 0;
\end{aligned}$$

We have found the information symbols $uk_6 \dots uk_{11}$ (coefficients corresponding to second-order vectors $x_i x_j$).

Let us define $uk_5 \dots uk_2$:

$$\begin{aligned}
uk_5 &= u_1 + u_9 = u_2 + u_{10} = u_3 + u_{11} = u_4 + u_{12} = u_5 + u_{15} = u_6 + u_{14} = u_7 + u_{15} = u_8 + u_{16} = 1 + 0 = 1; \\
uk_4 &= u_1 + u_5 = u_2 + u_6 = u_3 + u_7 = u_4 + u_8 = u_9 + u_{13} = u_{10} + u_{14} = u_{11} + u_{15} = u_{12} + u_{16} = 1 + 1 = 0; \\
uk_3 &= u_1 + u_3 = u_2 + u_4 = u_5 + u_7 = u_6 + u_8 = u_9 + u_{11} = u_{10} + u_{12} = u_{13} + u_{15} = u_{14} + u_{16} = 1 + 1 = 0; \\
uk_2 &= u_1 + u_2 = u_3 + u_4 = u_5 + u_6 = u_7 + u_8 = u_9 + u_{10} = u_{11} + u_{12} = u_{13} + u_{14} = u_{15} + u_{16} = 1 + 1 = 0;
\end{aligned}$$

To find uk_1 , it is necessary to calculate uk_i and x_i again and add them to the transformed message. If the result of the addition is a vector containing all ones, $uk_1 = 1$; if the resulting vector contains all zeros, $uk_1 = 0$. Thus as

$$uk_4 = uk_3 = uk_2 = 0,$$

then $0000000011111111 - uk_5 x_4$

1111111100000000-преобразованное сообщение

1111111111111111,

i.e. $uk_1 = 1$.

Thus, the original message extracted from the code word is 10001011011.

Independent work:

No	Task				
1	1. Decode the message:		2. Encode the message:		
	11111111111111 111 0101010101010 101 0011001100110 011 0000111100001 111 0000000011111 111 0001000100010 001 0000010100000 101 0000000001010 101 0000001100000 011 0000000000110 011 0000000000001 111 0010101101111 110		111111111111111 01010101010101 0011001100110011 0000111100001111 0000000011111111 0001000100010001 0000010100000101 0000000001010101 0000001100000011 0000000000110011 0000000000001111 ????????????????	0 0 0 1 0 0 0 1 1 1 1	
	2	1. Decode the message:		2. Decode the message:	
		111111111111111 0101010101010101 0011001100110011 0000111100001111 0000000011111111 0001000100010001 0000010100000101 0000000001010101 0000001100000011	? ? ? ? ? ? ? ? ?	111111111111111 0101010101010101 0011001100110011 0000111100001111 0000000011111111 0001000100010001 0000010100000101 0000000001010101 0000001100000011	? ? ? ? ? ? ? ? ?

	<table><tr><td>0000000000110011</td><td>?</td></tr><tr><td>0000000000001111</td><td>?</td></tr><tr><td>1111011000110101</td><td></td></tr></table>	0000000000110011	?	0000000000001111	?	1111011000110101			<table><tr><td>0000000000110011</td><td>?</td></tr><tr><td>0000000000001111</td><td>?</td></tr><tr><td>1110101101110010</td><td></td></tr></table>	0000000000110011	?	0000000000001111	?	1110101101110010																																																																		
0000000000110011	?																																																																															
0000000000001111	?																																																																															
1111011000110101																																																																																
0000000000110011	?																																																																															
0000000000001111	?																																																																															
1110101101110010																																																																																
3	<table><tr><td colspan="2">1. Decode the message:</td></tr><tr><td>11111111111111</td><td>?</td></tr><tr><td>01010101010101</td><td>?</td></tr><tr><td>0011001100110011</td><td>?</td></tr><tr><td>0000111100001111</td><td>?</td></tr><tr><td>0000000011111111</td><td>?</td></tr><tr><td>0001000100010001</td><td>?</td></tr><tr><td>0000010100000101</td><td>?</td></tr><tr><td>0000000001010101</td><td>?</td></tr><tr><td>0000001100000011</td><td>?</td></tr><tr><td>0000000000110011</td><td>?</td></tr><tr><td>0000000000001111</td><td>?</td></tr><tr><td>1001001111110101</td><td></td></tr></table>	1. Decode the message:		11111111111111	?	01010101010101	?	0011001100110011	?	0000111100001111	?	0000000011111111	?	0001000100010001	?	0000010100000101	?	0000000001010101	?	0000001100000011	?	0000000000110011	?	0000000000001111	?	1001001111110101			<table><tr><td colspan="2">2. Decode the message:</td></tr><tr><td>1111111111111111</td><td>?</td></tr><tr><td>0101010101010101</td><td>?</td></tr><tr><td>0011001100110011</td><td>?</td></tr><tr><td>0000111100001111</td><td>?</td></tr><tr><td>0000000011111111</td><td>?</td></tr><tr><td>0001000100010001</td><td>?</td></tr><tr><td>0000010100000101</td><td>?</td></tr><tr><td>0000000001010101</td><td>?</td></tr><tr><td>0000001100000011</td><td>?</td></tr><tr><td>0000000000110011</td><td>?</td></tr><tr><td>0000000000001111</td><td>?</td></tr><tr><td>0110111111000101</td><td></td></tr></table>	2. Decode the message:		1111111111111111	?	0101010101010101	?	0011001100110011	?	0000111100001111	?	0000000011111111	?	0001000100010001	?	0000010100000101	?	0000000001010101	?	0000001100000011	?	0000000000110011	?	0000000000001111	?	0110111111000101																										
1. Decode the message:																																																																																
11111111111111	?																																																																															
01010101010101	?																																																																															
0011001100110011	?																																																																															
0000111100001111	?																																																																															
0000000011111111	?																																																																															
0001000100010001	?																																																																															
0000010100000101	?																																																																															
0000000001010101	?																																																																															
0000001100000011	?																																																																															
0000000000110011	?																																																																															
0000000000001111	?																																																																															
1001001111110101																																																																																
2. Decode the message:																																																																																
1111111111111111	?																																																																															
0101010101010101	?																																																																															
0011001100110011	?																																																																															
0000111100001111	?																																																																															
0000000011111111	?																																																																															
0001000100010001	?																																																																															
0000010100000101	?																																																																															
0000000001010101	?																																																																															
0000001100000011	?																																																																															
0000000000110011	?																																																																															
0000000000001111	?																																																																															
0110111111000101																																																																																
4	<table><tr><td colspan="2">1. Encode the message:</td></tr><tr><td>11111111111111</td><td>1</td></tr><tr><td>01010101010101</td><td>0</td></tr><tr><td>0011001100110011</td><td>0</td></tr><tr><td>0000111100001111</td><td>1</td></tr><tr><td>0000000011111111</td><td>0</td></tr><tr><td>0001000100010001</td><td>0</td></tr><tr><td>0000010100000101</td><td>0</td></tr><tr><td>0000000001010101</td><td>1</td></tr><tr><td>0000001100000011</td><td>0</td></tr><tr><td>0000000000110011</td><td>0</td></tr><tr><td>0000000000001111</td><td>1</td></tr><tr><td>??????????????</td><td></td></tr><tr><td>????</td><td></td></tr></table>	1. Encode the message:		11111111111111	1	01010101010101	0	0011001100110011	0	0000111100001111	1	0000000011111111	0	0001000100010001	0	0000010100000101	0	0000000001010101	1	0000001100000011	0	0000000000110011	0	0000000000001111	1	??????????????		????			<table><tr><td colspan="2">2. Decode the message:</td></tr><tr><td>1111111111111111</td><td>?</td></tr><tr><td>0101010101010101</td><td>?</td></tr><tr><td>0011001100110011</td><td>?</td></tr><tr><td>0000111100001111</td><td>?</td></tr><tr><td>0000000011111111</td><td>?</td></tr><tr><td>0001000100010001</td><td>?</td></tr><tr><td>0000010100000101</td><td>?</td></tr><tr><td>0000000001010101</td><td>?</td></tr><tr><td>0000001100000011</td><td>?</td></tr><tr><td>0000000000110011</td><td>?</td></tr><tr><td>0000000000001111</td><td>?</td></tr><tr><td>1110011101000010</td><td></td></tr></table>	2. Decode the message:		1111111111111111	?	0101010101010101	?	0011001100110011	?	0000111100001111	?	0000000011111111	?	0001000100010001	?	0000010100000101	?	0000000001010101	?	0000001100000011	?	0000000000110011	?	0000000000001111	?	1110011101000010																								
1. Encode the message:																																																																																
11111111111111	1																																																																															
01010101010101	0																																																																															
0011001100110011	0																																																																															
0000111100001111	1																																																																															
0000000011111111	0																																																																															
0001000100010001	0																																																																															
0000010100000101	0																																																																															
0000000001010101	1																																																																															
0000001100000011	0																																																																															
0000000000110011	0																																																																															
0000000000001111	1																																																																															
??????????????																																																																																
????																																																																																
2. Decode the message:																																																																																
1111111111111111	?																																																																															
0101010101010101	?																																																																															
0011001100110011	?																																																																															
0000111100001111	?																																																																															
0000000011111111	?																																																																															
0001000100010001	?																																																																															
0000010100000101	?																																																																															
0000000001010101	?																																																																															
0000001100000011	?																																																																															
0000000000110011	?																																																																															
0000000000001111	?																																																																															
1110011101000010																																																																																
5	<table><tr><td colspan="2">1. Encrypt the message: 111111111111</td></tr><tr><td>111</td><td></td></tr><tr><td>0101010101010101</td><td></td></tr><tr><td>101</td><td></td></tr><tr><td>0011001100110</td><td></td></tr><tr><td>011</td><td></td></tr><tr><td>0000111100001</td><td></td></tr><tr><td>111</td><td></td></tr><tr><td>0000000011111</td><td></td></tr><tr><td>111</td><td></td></tr><tr><td>0001000100010</td><td></td></tr><tr><td>001</td><td></td></tr><tr><td>0000010100000</td><td></td></tr><tr><td>101</td><td></td></tr><tr><td>0000000001010</td><td></td></tr><tr><td>101</td><td></td></tr><tr><td>0000001100000</td><td></td></tr><tr><td>011</td><td></td></tr><tr><td>0000000000110</td><td></td></tr></table>	1. Encrypt the message: 111111111111		111		0101010101010101		101		0011001100110		011		0000111100001		111		0000000011111		111		0001000100010		001		0000010100000		101		0000000001010		101		0000001100000		011		0000000000110			<table><tr><td colspan="2">2. Encrypt the message:</td></tr><tr><td>1111111111111111</td><td></td></tr><tr><td>11</td><td></td></tr><tr><td>01010101010101</td><td></td></tr><tr><td>01</td><td></td></tr><tr><td>00110011001100</td><td></td></tr><tr><td>11</td><td></td></tr><tr><td>00001111000011</td><td></td></tr><tr><td>11</td><td></td></tr><tr><td>00000000111111</td><td></td></tr><tr><td>11</td><td></td></tr><tr><td>00010001000100</td><td></td></tr><tr><td>01</td><td></td></tr><tr><td>00000101000001</td><td></td></tr><tr><td>01</td><td></td></tr><tr><td>00000000010101</td><td></td></tr><tr><td>01</td><td></td></tr><tr><td>00000011000000</td><td></td></tr><tr><td>11</td><td></td></tr></table>	2. Encrypt the message:		1111111111111111		11		01010101010101		01		00110011001100		11		00001111000011		11		00000000111111		11		00010001000100		01		00000101000001		01		00000000010101		01		00000011000000		11		
1. Encrypt the message: 111111111111																																																																																
111																																																																																
0101010101010101																																																																																
101																																																																																
0011001100110																																																																																
011																																																																																
0000111100001																																																																																
111																																																																																
0000000011111																																																																																
111																																																																																
0001000100010																																																																																
001																																																																																
0000010100000																																																																																
101																																																																																
0000000001010																																																																																
101																																																																																
0000001100000																																																																																
011																																																																																
0000000000110																																																																																
2. Encrypt the message:																																																																																
1111111111111111																																																																																
11																																																																																
01010101010101																																																																																
01																																																																																
00110011001100																																																																																
11																																																																																
00001111000011																																																																																
11																																																																																
00000000111111																																																																																
11																																																																																
00010001000100																																																																																
01																																																																																
00000101000001																																																																																
01																																																																																
00000000010101																																																																																
01																																																																																
00000011000000																																																																																
11																																																																																

	011			00000000001100				
	00000000000001			11				
	111			000000000000011				
	??????????????			11				
	???			????????????????				
				??				

Review questions:

- 1) What are the characteristics of Reed-Maller codes?
- 2) How are Reed-Maller codes constructed?
- 3) How is the Reed-Maller code decoded?

LABORATORY WORK No. 12

Cyclic codes

These codes are widely used due to their simple hardware implementation and high correction capabilities. These codes got their name from their property: any cyclic shift of a valid word results in a valid code word.

For example, if the code word is $C_1 C_2 C_3 \dots C_{n-1} C_n$ is valid, then the code word $C_n C_1 C_2 C_3 \dots C_{n-1}$ is also valid.

For constructing and analysis cyclic codes uses a polynomial representation of binary sequences.

Representation of binary codes as polynomials.

A polynomial is a mathematical expression

$$f(x) = f_n * x^n + f_{n-1} * x^{n-1} + \dots + f_1 * x + f_0$$

Where $f_i \in (0,1)$ (binary case), x is a pseudo-variable (fictitious).

Example.

$$\begin{array}{ccccccc} & 6 & 5 & 4 & 3 & 2 & 1 & 0 \\ \text{A:} & 1 & 0 & 1 & 0 & 1 & 1 & 1 \end{array}$$

$$A(x) = x^6 + x^4 + x^2 + x + 1$$

Operations on polynomials

1. Addition/Subtraction

For binary numbers, these operations are equivalent.

$$\begin{array}{r} a \ b \ c \\ 0 \ 0 \ 0 \\ 0 \ 1 \ 1 \\ 1 \ 0 \ 1 \\ 1 \ 1 \ 0 \end{array}$$

Example:

$$\begin{array}{r} A(x) = x^6 + x^4 + x^2 + x + 1 \\ \pm B(x) = x^7 + x^5 + x^4 + x^3 + x + 1 \\ \hline C(x) = x^7 + x^6 + x^5 + x^3 + x^2 \end{array}$$

2. Multiplication of polynomials.

Example:

$$\begin{array}{r} * \quad A(x) = x^5 + x^2 + 1 \\ \quad B(x) = x^2 + x + 1 \\ \hline \quad x^5 + x^2 + 1 \\ + \quad x^6 + x^3 + x \\ + \quad x^7 + x^4 + x^2 \\ \hline C(x) = x^7 + x^6 + x^5 + x^4 + x^3 + x + 1 \end{array}$$

Division of polynomials.

Example:

$$\begin{array}{r}
 C(x) = x^7 + x^6 + x^5 + x^4 + x^3 + x + 1 \\
 A(x) = x^5 + x^2 + 1 \\
 \hline
 x^7 + x^6 + x^5 + x^4 + x^3 + x + 1 \quad | \quad x^5 + x^2 + 1 \\
 - x^7 + x^4 + x^2 \\
 \hline
 x^6 + x^5 + x^3 + x^2 + x + 1 \quad | \quad x^2 + x + 1 \\
 - x^6 + x^3 + x \\
 \hline
 x^5 + x^2 + 1 \\
 - x^5 + x^2 + 1 \\
 \hline
 0
 \end{array}$$

$R_{A(x)}[C(x)] = 0$ – remainder from dividing polynomial $C(x)$ by polynomial $A(x)$
 $A(x) = x^5 + x^2 + 1$

$$\begin{array}{r}
 B(x) = x^3 + 1 \\
 x^5 + x^2 + 1 \quad | \quad x^3 + 1 \\
 - x^5 + x^2 \\
 \hline
 1 \quad | \quad x^2
 \end{array}$$

210
R=001

The remainder of the division must be less than the degree of the divisor $B(x)$.

The degree of a polynomial is the maximum degree of pseudo-variables with non-zero coefficients.

$$\deg A(x) = 5; \deg B(x) = 3$$

Independent work:

Option	Tasks
1	1. What is the form of a cyclic code combination, the information part of which is $U(x) = 1011$ and the generating polynomial $K(x) = 1101$?
	2. Construct the complete generator matrix of the cyclic code using the following irreducible polynomial: $x^4 + x + 1$.
2	1. Construct a cyclic code with minimum code distance $d_0=3$ for transmitting 16 messages.
	2. Determine the total length of the code combination of the cyclic code with $d_0=3$, if the number of information bits is $n_i = 5$.
3	1. Determine the possible number of information bits in a 15-character code that detects two errors or corrects one error.
	2. Detect a fivefold error in the message $F(x)=100011101101011$, if it is known that the code was constructed using binomial $(x+1)$.
4	1. Construct a cyclic code with a length of 15 characters that corrects one or two errors.
	2. Show the process of constructing a cyclic code for transmitting 50 messages. Provide the possibility of correcting a single error and check the corrective capabilities of the code on one of the combinations obtained.
5	1. Check the possibility of correcting the error in the 30th and 11th combinations of the code obtained in the previous task.

2. Can the irreducible polynomial x^5+x^2+1 be used as a generator for constructing a cyclic code with a minimum code distance $d_0=5$?
--

Review questions:

- 1) What are cyclic codes?
- 2) What is the principle of constructing cyclic codes?
- 3) What is the matrix representation of cyclic codes?

LABORATORY WORK No. 13

Hamming cyclic codes

Construction of cyclic codes

Generating (forming) polynomials play a central role in the construction of cyclic codes. For Hamming cyclic codes, *irreducible polynomials* are used as generating polynomials.

An irreducible polynomial is one that cannot be represented as a product of polynomials of lower degrees (analogous to prime numbers). Irreducible polynomials are selected from tables.

The number of check symbols in a cyclic code is equal to the degree of the generating polynomial, and, conversely, the degree of the generating polynomial is equal to the number of check symbols.

Hamming cyclic codes

There are systematic and unsystematic cyclic codes.

The basic idea of cyclic coding is that the allowed code word must be divisible by the generator polynomial without a remainder.

The remainder from dividing the code word received by the decoder by the generator polynomial acts as a syndrome. If the remainder is zero, the decoder assumes that there were no errors. In the case of a non-zero syndrome, depending on its type, the decoder performs error correction.

Construction of a systematic cyclic Hamming code.

Example. Let, for example, the information symbols be equal to the combination 1101. In this case, $k = 4$. For Hamming codes, the number of correctable is equal to $s=1$, then $d_{\min}=3$.

The polynomial representation of information symbols has the form

$$A(x)=x^3+x^2+1.$$

Using the known formula, we determine the number of check symbols:

$$p=[\log_2 (k+1)+[\log_2 (k+1)]]=3.$$

Code length:

$$n= k+ p= 7.$$

$\deg K(x) = p = 3$ degree of the generating polynomial.

$K(x)=x^3+x+1$ (selected from the table of generating polynomials).

The encoder of the systematic cyclic Hamming code must perform multiply the information sequence by the polynomial x^p and division by the generating polynomial. The resulting remaining from division represents check symbols.

$$A(x) * x^p + R_{K(x)}[A(x) * x^p]$$

$$\begin{array}{r}
 A(x) * x^p = (x^3+x+1) * x^3 = x^6 + x^5 + x^3 \quad \left| \begin{array}{l} x^3 + x + 1 \\ x^3 + x^2 + x + 1 \end{array} \right. \\
 \underline{-x^6 + x^4 + x^3} \\
 x^5 + x^4 \\
 \underline{-x^5 + x^3 + x^2} \\
 x^4 + x^3 + x^2 \\
 \underline{-x^4 + x^2 + x} \\
 x^3 + x \\
 \underline{-x^3 + x + 1} \\
 1 \rightarrow 001
 \end{array}$$

We obtained a systematic $x^6 + x^5 + x^3 + 1 = 1101\ 001$, where the first part 1101 is informational, and the second part 001 is check symbols.

Thus, the resulting code word must be divisible by the generating polynomial without remainder:

$$\begin{array}{r}
 x^6 + x^5 + x^3 + 1 \quad \left| \begin{array}{l} x^3 + x + 1 \\ x^3 + x^2 + x + 1 \end{array} \right. \\
 \underline{-x^6 + x^4 + x^3} \\
 x^5 + x^4 + 1 \\
 \underline{-x^5 + x^3 + x^2} \\
 x^4 + x^3 + x^2 + 1 \\
 \underline{-x^4 + x^2 + x} \\
 x^3 + x + 1 \\
 \underline{-x^3 + x + 1} \\
 0
 \end{array}$$

A systematic cyclic code decoder divides the received code word by the generator polynomial and, depending on the type of syndrome (division remainder), corrects the code word.

Matrix representation of the Hamming cyclic code. Systematic cyclic code.

Example. Let, for example, the number of information symbols be $k = 4$. For Hamming codes, the number of correctable errors is equal to $s = 1$, then $d_{\min} = 3$.

Using the known formula, we determine the number of check symbols:

$$p = \lceil \log_2 (k+1) \rceil + \lceil \log_2 (k+1) \rceil = 3.$$

Code length:

$$n = k + p = 7.$$

$\deg K(x) = p = 3$ degree of the generating polynomial.

$K(x) = x^3 + x + 1$ (selected from the table of generating polynomials).

Generating matrix of the Hamming systematic code:

$$P_{(n,k)} = IH_p$$

where I is a unit matrix with k rows and k columns;

H_p is a check submatrix, which is a matrix of k rows and p columns. Each row represents check symbols for the corresponding combination of information symbols of the unit matrix.

The generator matrix allows us to obtain k valid code combinations. The remaining valid combinations are obtained by summing the modulus two rows of the generator matrix in all possible combinations.

Determination of check symbols for constructing the generating matrix:

1. $k = 0001$

$$\begin{array}{r|l} x^3 & x^3 + x + 1 \\ -x^3 + x + 1 & 1 \\ \hline & x + 1 \end{array}$$

$$\begin{array}{c} 2 \ 1 \ 0 \\ p = 011 \end{array}$$

2. $k = 0010$

$$\begin{array}{r|l} x^4 & x^3 + x + 1 \\ -x^4 + x^2 + x & x \\ \hline & x^2 + 1 \end{array}$$

$$\begin{array}{c} 2 \ 1 \ 0 \\ p = 110 \end{array}$$

3. $k = 0100$

$$\begin{array}{r|l}
 x^5 & x^3 + x + 1 \\
 -x^5 + x^3 + x^2 & x^2 + 1 \\
 \hline
 x^3 + x^2 & \\
 -x^3 + x + 1 & \\
 \hline
 x^2 + x + 1 &
 \end{array}$$

$$\begin{array}{r}
 210 \\
 p = 111
 \end{array}$$

4. $k = 1000$

$$\begin{array}{r|l}
 x^6 & x^3 + x + 1 \\
 -x^6 + x^4 + x^3 & x^3 + x + 1 \\
 \hline
 x^4 + x^3 & \\
 -x^4 + x^2 + x & \\
 \hline
 x^3 + x + 1 & \\
 -x^3 + x + 1 & \\
 \hline
 x^2 + 1 &
 \end{array}$$

$$\begin{array}{r}
 210 \\
 p = 101
 \end{array}$$

Generating matrix for the systematic Hamming code (7,4):

$$P = \begin{array}{c|cc|c}
 3 & 2 & 1 & 0 \\
 \hline
 1000 & 101 & & \\
 0100 & 111 & & \\
 0010 & 110 & & \\
 0001 & 011 & &
 \end{array}$$

For specific information symbols, to determine the code word, it is necessary to add the two rows of the generating matrix, whose numbers are equal to the numbers of the positions of the units in the combination of information symbols, character by character. For example, for the information symbols 1101, it is necessary to add the first, second, and fourth rows. We obtain the code word 1101001.

Independent work:

Option	Tasks
--------	-------

1	1. Encode the message with code $d_{min} = 4$: 0101000110. The generating polynomial for the code with $d_{min} = 3$ is 10011.
	2. Accepted combination (code with $d_{min} = 4$): 00101101010011. The generating polynomial for the code with $d_{min} = 3$ is 10011.
2	1. Accepted combination (code with $d_{min} = 4$): 100001001110001. The generating polynomial for the code with $d_{min} = 3$ is 10011.
	2. Encode the message with code with $d_{min} = 4$: 1011111101. The generating polynomial for code with $d_{min} = 3$ is 10011.
3	1. Encode the message with code $d_{min} = 4$: 110011101. The generating polynomial for code $d_{min} = 3$ is 10011.
	2. The received combination (code with $d_{min} = 4$): 111011110111000. The generating polynomial for the code with $d_{min} = 3$ is 10011.
4	1. Accepted combination (code with $d_{min} = 4$): 110100111100001. The generating polynomial for the code with $d_{min} = 3$ is 10011.
	2. Encode the message with code with $d_{min} = 4$: 1101110110. The generating polynomial for code with $d_{min} = 3$ is 10011.
5	1. Accepted combination (code with $d_{min} = 4$): 000110111000111. The generating polynomial for the code with $d_{min} = 3$ is 10011.
	2. Encode the message with code with $d_{min} = 4$: 1010111100. The generating polynomial for code with $d_{min} = 3$ is 10011.

Review questions:

- 1) Which codes are capable of correcting single errors?
- 2) What are the capabilities of the Hamming cyclic code?
- 3) What is a generating polynomial?

LABORATORY WORK No. 14

Bose-Choudhury-Hawking Codes (BCH)

BCH codes, developed by Bowes, Choudhury, and Hawking, allow any number of errors to be detected and corrected. The given values in coding are: the number of errors s to be corrected and the total number of symbols sent to the line, i.e., the word length n . The number of information symbols (k) and check symbols (m), as well as the composition of the check symbols, are to be determined.

Coding algorithm.

1. Selecting the word length. When encoding using the BCH method, the word length n cannot be chosen arbitrarily. The first restriction is that a word can only have an odd number of symbols. The following equality must be satisfied: $2^h - 1 = n$, where h is an integer.

2. Determination of the code distance d . $d = 2s + 1$.

3. Determination of the generating polynomial $P(X)$. The generating polynomial is the least common multiple (LCM) of the so-called minimal

polynomials $M(X)$ up to and including order $2s-1$. Moreover, the generating polynomial is composed of the product of a certain number of odd minimal polynomials $P(X) = LCM[M_{(1)}(X) \cdot M_3(X) \cdot \dots \cdot M_{2s-1}(X)]$. (1) Minimal polynomials are simple irreducible polynomials. Note that if there are two identical minimal polynomials, one of them is excluded.

4. Determining the number of minimal polynomials (L). From point 3, it follows that the order of the minimal polynomials is defined as $2s-1$. It can be shown that the number of minimal polynomials is equal to the number of correctable errors, i.e., $L = s$.

5. Determination of the highest degree l of the minimal polynomial. The degree l is the smallest integer for which $2-1$ is divisible by n . This means that $l = h$.

6. Selection of minimal polynomials. Once the number of minimal polynomials L and the degree of the highest polynomial l have been determined, the polynomials are written out from Table 1.

7. Determination of the degree b of the generating polynomial $P(X)$. The degree of the generating polynomial does not exceed the product $l \cdot s$ or $L \cdot s$. Depending on the LCM , the degree $P(X)$ may be less than $b \leq l \cdot s$. After finding all the minimal polynomials, the generating polynomial is found by equation (1).

8. Determining the number of check characters. Since the number of check characters m is equal to the degree of the generating polynomial, for a code of length n , $b=m \leq l \cdot s$ is satisfied.

9. Determination of number information of is determined from the equation $k=n-m$.

10. Calculation of the encoded message. The encoded message $F(X)$ can be obtained in two ways:

1. $F(X)=Q(X) \cdot P(X)$, where $P(X)$ is the generating polynomial, and $Q(X)$ is obtained as follows:

$$\frac{x^m G(x)}{P(x)} = Q(x) + \frac{R(x)}{P(x)}$$

where $G(X)$ is the combination to be encoded;
 $R(X)$ is the remainder from division.

2. $F(X)=x^m \cdot G(X) + R(X)$.

Encoding example

A message must be encoded with the ability to correct 2 errors so that 15 bits are transmitted to the data channel ($n=15$, $h=4$).

The code distance is $d = 2s + 1 = 5$.

Two minimal polynomials will be used. The highest degree of the minimal polynomial is 4.

We use the minimal polynomials 10011 and 1011.

Then the generating polynomial is equal to $10011 \cdot 1011 = 10101101$, i.e., 15 bits of the 7 code, 5 will be check bits.

Encoding a specific message with the BCH code is no different from encoding with other cyclic codes. An example of obtaining an encoded combination from the original message and the generating polynomial.

Decoding

Decoding and error correction of the BCH code is performed in the same way as for any other cyclic code.

Note. The following code parameters are assumed for testing: 15 bits are transmitted to the channel and 2 errors are corrected.

Table 5

L	Minimal irreducible polynomials of various degrees						
	2	3	4	5	6	7	8
1	111	1011	10011	100101	1000011	10001001	100011101
3		1101	11111	111101	1010111	10001111	101110111
5			111	110111	1100111	10011101	111110011
7			11001	101111	1001001	11110111	101101001
9				110111	1101	10111111	110111101
11				111011	1101101	110101	111100011
13						10000011	100101011

Independent work:

Option No.	Tasks
------------	-------

1	1. Encrypt the message: 00000001.
	2. Received combination: 100111111100110. Restore the original message.
2	1. Accepted combination: 011010101101011. Restore the original message.
	2. Encode the message: 01111011.
3	1. Encode the message: 11001111.
	2. Accepted combination: 100100110010110. Restore the original message.
4	1. Accepted combination: 100010110100000. Restore the original message.
	2. Encode the message: 01011001.
5	1. Accepted combination: 011110111110011. Restore the original message.
	2. Encode the message with the code: 10000001. Restore the original message.

Review questions:

- 1) What methods exist for finding the generating polynomial for the BCH code?
- 2) How is the length of the BCH code combination found?
- 3) What are the properties of the BCH code?

LABORATORY WORK No. 15

Abramson Code

Cyclic codes are block systematic codes in which each combination is encoded independently (as a block) in such a way that the information (k) and control (m) symbols are always in specific places. Cyclic coding is based on the use of an irreducible polynomial $P(x)$, which, in the context of cyclic codes, is called a generator polynomial. Combinations of binary code for all combinations are taken as information symbols for constructing cyclic codes. A cyclic code or encoded message $F(x)$ can be formed in two ways:

1. By multiplying one of the combinations of the binary code for all combinations (combination $Q(x)$), belonging to the same group of the same code as the given one (combination $G(x)$), by the generating polynomial $P(x)$;
2. Multiplying the given code combination $G(x)$ by a monomial x^m , which has the same degree as the generating polynomial $P(x)$, adding to this product the remainder from dividing the product $x^m \cdot G(x)$ by the generating polynomial $P(x)$. Thus, $F(x) = Q(x) \cdot P(x) = (x^m) \cdot G(x) + R(x)$.

Abramson codes belong to a group of cyclic codes that allow single error bursts to be detected and corrected. Consequently, the encoding procedure for Abramson codes is similar to the standard encoding procedure for cyclic codes. The length of the code combination for Abramson codes is $n=2^{(h)}-1$, where h is the degree of the irreducible

polynomial $P_1(x)$. The generating polynomial is $P(x) = (1+x) \cdot P_1(x)$. The number of information symbols $k = 2^h - h - 2$. These expressions are valid for the minimum code distance $d_{min} = 4$, which allows all single and all adjacent double errors to be corrected.

Coding example

The message *0110010110* needs to be encoded.

Given the irreducible polynomial $P_1 = 10011$, i.e. $P_1(x) = x^4 + x + 1$. The generating polynomial in this case will be equal to $P(x) = (1+x) \cdot (x^4 + x + 1) = x^5 + x^4 + x^2 + 1$. We need to find the remainder of the division of $G(x) \cdot x^5$ by the generating polynomial $x^5 + x^4 + x^2 + 1$:

```

0 1 1 0 0 1 0 1 1 0 0 0 0 0 0 | 1 1 0 1 0 1
  1 1 0 1 0 1                      -----
-----
    1 1 1 1 1 0
    1 1 0 1 0 1
    -----

```

```

1 0 1 1 0 0
1 1 0 1 0 1
-----
1 1 0 0 1 0
1 1 0 1 0 1
-----
1 1 1 0 0

```

Thus, the encoded message $F(x) = (x^m) \cdot G(x) + R(x) = 011001011011100$. The verification part is highlighted in italics.

Decoding

The idea behind detecting errors in the received cyclic code is that, in the absence of errors, the encoded combination $F(x)$ is divisible by the generating polynomial $P(x)$ without a remainder. In this case, the check symbols are discarded and the information symbols are used as intended. If the received code combination is distorted, this combination $F(x)$ is converted into a combination $H(x)$, which can be represented as the sum of two polynomials: $H(x) = F(x) + E(x)$, where $E(x)$ is an error polynomial containing as many units as there are elements of the received combination that do not match the elements in the transmitted combination. Cyclic codes can be decoded as follows:

1. Calculate the remainder (syndrome) $R(x)$. If $R(x)=0$, then the combination is accepted without errors. The presence of a remainder indicates that the combination is accepted as distorted.

2. Calculate the weight of the remainder W . If $W \leq s$ (s is the number of correctable errors), then the received combination is added modulo 2 to the remainder to obtain the corrected combination.

3. If $W > s$, then a cyclic shift of one character to the left is performed, and the combination obtained after such a shift is again divided by the generating polynomial. If after this $W \leq s$, then the cyclically shifted combination is added to the remainder and then, after addition, shifted back one character to the right (returned to its previous place). As a result, we obtain the corrected combination.

4. If, after the cyclic shift by one character, W is still *greater than* s , additional cyclic shifts to the left are performed. After each shift, the resulting combination is divided by $P(X)$ and the weight of the remainder is checked. If $W \leq s$, the actions specified in paragraph 3 are performed, with the only difference being that the number of reverse cyclic shifts to the right must be equal to the number of shifts to the left.

Decoding example

Encoded (received) combination: *00011111110110*. Irreducible polynomial $P_1 = 10011 = x^4 + x + 1$.

The generating polynomial is found as follows: $P(x) = (1+x) \cdot (x^4 + x + 1) = x^5 + x^4 + x^2 + 1$.

We need to find the remainder of the division of the received message by the generating polynomial:

```

000111111110110|110101
  110101          -----
-----
  101011
  110101
-----
  111100
  110101
-----
  100111
  110101
-----
  100100
  110101
-----
  10001

```

The weight of the remainder is 2. Since Abramson's code is capable of correcting all single and adjacent double errors (in this case, the single bits of the remainder are not adjacent), it is necessary to perform a cyclic shift of the initial message by one bit to the left and repeat the division:

```

0011111111101100|110101
  110101          -----
-----
  101011
  110101
-----
  111100
  110101
-----
  100111
  110101
-----
  100100
  110101
-----
  100010
  110101
-----
  10111

```

Weight Balance equal 4, means it is necessary continue performing to cyclic shift and division.

```

01111111111011000|110101
  110101          -----
-----
  101011
  110101
-----
  111100
  110101
-----
  10011

```

```

1 1 0 1 0 1
-----
1 0 0 1 0 0
1 1 0 1 0 1
-----
1 0 0 0 1 0
1 1 0 1 0 1
-----
1 0 1 1 1 0
1 1 0 1 0 1
-----
1 1 0 1 1

```

Weight Balance equal 4, means it is necessary continue performing to cyclic shift and division.

```

1 1 1 1 1 1 1 1 0 1 1 0 0 0 0 | 1 1 0 1 0 1
1 1 0 1 0 1
-----
1 0 1 0 1 1
1 1 0 1 0 1
-----
1 1 1 1 0 0
1 1 0 1 0 1
-----
1 0 0 1 1
1 1 0 1 0 1
-----
1 0 0 1 0 0
1 1 0 1 0 1
-----
1 0 0 0 1 0
1 1 0 1 0 1
-----
1 0 1 1 1 0
1 1 0 1 0 1
-----
1 1 0 1 1 0
1 1 0 1 0 1
-----
1 1

```

In this case, the weight of the remainder is 2, and the bits to be corrected are adjacent. To obtain the corrected combination, it is necessary to add the shifted combination with the remainder modulo 2 (the result is 111111110110011) and shift the result cyclically three positions to the right (011111111110110). Discarding the check part, we get the final result – 011111111.

Independent work:

Option No.	Tasks
	1. Encode the message: 0001010011. Irreducible polynomial P1: 11101.

1	2. The encoded combination 110011001000010. Known irreducible polynomial P1: 11101. Restore the original message.
2	1. Encrypt the message: 1100001111. Irreducible polynomial P1: 11101.
	2. The encoded combination 110010001101011. Known irreducible polynomial P1: 11101. Restore the original message.
3	1. Encrypt the message: 1001111001. Irreducible polynomial P1: 11101.
	2. The encoded combination 110111100010100. Known irreducible polynomial P1: 11101. Restore the original message.
4	1. Encrypt the message: 1100101011. Irreducible polynomial P1: 11101.
	2. The encoded combination 001100110001111. Known irreducible polynomial P1: 11101. Restore the original message.
5	1. Encrypt the message: 1100101111. Irreducible polynomial P1: 11101.
	2. The encoded combination 110111100010100. Known irreducible polynomial P1: 11101. Restore the original message.

Test questions:

- 1) What is the length of the code combination of Abramson codes?
- 2) What are the capabilities of the Abramson cyclic code?
- 3) Describe the principle of encoding with the Abramson code.